Branch Coverage Report

Blood Bank Management System (BBMS) v1.0 - Backend

Project	SE Mini Project - Team 9, Blood Bank Management System
Repository	blood-bank-management-team-9/se-mini-project-team9-blood-bank
Branch tested	feature/18-record-donation (last confirmed remote commit 60a5136)
Related issue	GitHub Issue #71 - Upload Branch Coverage Report
Governing document	BBMS Software Test Plan v1.0 (03-10-2026), Test Execution Cycle 2: 02-11-2026 to 06-11-2026
Coverage run date	5 November 2026
Prepared by	Test Engineer (Chukkapalli Rohan)
Status	Draft for QA Lead review

96.48% Branch coverage (466 of 483)	**365** passed **41** skipped **0** failed	**17** uncovered branches across 5 modules

1. Introduction

This report presents the branch coverage achieved by the automated backend unit test suite of the Blood Bank Management System (BBMS). It is one of four test reports required by the project's testing assignment (Unit Test Report, System Validation Report, Code Coverage Report and Branch Coverage Report) and is prepared under the BBMS Software Test Plan v1.0 (STP), which follows IEEE Std 829-1998.

Scope. The report covers the Node.js backend source under `backend/src`, as measured by the Jest coverage run of 5 November 2026. It does not cover the front end, performance, security, usability or system-level behaviour; those are addressed by other test types in the STP.

What branch coverage measures. A branch is one possible outcome of a decision point in the code, such as the true and false outcomes of an `if` statement, each case of a `switch`, a ternary operator, a short-circuit expression (`&&`, `||`, `??`) or a default parameter value. Branch coverage is the percentage of these outcomes that the tests exercised at least once. It is stricter than statement or line coverage, because a statement can be executed without ever exercising the alternative path of the decision that guards it.

2. Alignment with the Software Test Plan

The STP does not set a numeric target for branch coverage specifically. The only coverage criterion it defines is in the quality requirement and test case below.

STP item	Requirement	Relevance to this report
NFR-QUAL-04 (Maintainability and testability)	Code organised in per-feature modules; unit tests cover at least 70% of business logic.	Measured coverage is well above 70% on every metric. Per-feature modules are visible in the module breakdown (Section 6).
TC-37, step 2	Run the unit test suite with a coverage report. Expected: unit tests cover at least 70% of business logic.	This report and the supporting HTML coverage report are the evidence for step 2.
TC-37, step 1 (NFR-QUAL-02)	Monitor uptime over a calendar month; availability at least 99%.	**Not covered here.** This needs separate evidence, so TC-37 should not be marked fully passed on the basis of this report alone.
STP Section 5, Unit tests	Business logic such as eligibility calculation, expiry date calculation and stock decrement, using Jest.	Matches the tooling and scope of the suite measured here.
STP Section 5, Exit criteria	100% of TC-01 to TC-37 executed; 0 critical or high defects open; RTM complete.	Coverage is one input to these criteria. It does not show that all 37 test cases were executed.

Because the STP's 70% criterion does not name a metric, this report treats the 70% threshold as applying to branch coverage as well as to statements, functions and lines. All four measured values exceed it. The STP also describes the threshold as applying to business logic; the figures reported here are for the whole backend source measured by the Jest run, which includes infrastructure code such as the database layer.

3. Methodology and Tools

- **Test framework:** Jest, as specified in STP Section 6 (Tools).
- **Coverage instrumentation:** Jest's built-in coverage collection with the standard HTML (lcov-report) reporter, which reports statement, branch, function and line coverage.

- **Command used:** `npm.cmd test -- --coverage`, run from the `backend` directory on Windows PowerShell.
- **Test style:** Unit tests, with database-dependent tests skipped where no database was configured (see Section 8). Test results below come from tests that ran against mocks or isolated test setups.
- **Test additions for this cycle:** 17 test files were added or modified and pushed to `feature/18-record-donation` (4 modified, 13 new), covering audit retention, donation repositories, donor service and repository, admin, authentication, CSRF and error-handling middleware, and stock repository and routes. These are testing changes only; they are not new application features.

4. Test Environment

Item	Detail
Machine	Developer laptop, Windows, PowerShell (local repository at the team member's workstation)
Source branch	feature/18-record-donation
Database	Not available in this run: DATABASE_URL, DB_HOST, DB_NAME and DB_USER were not configured, no local PostgreSQL service was detected, and the Docker Desktop Linux engine was unavailable.
Browsers / system tests	Not applicable to this report.

5. Test Execution Summary

The full run on 5 November 2026 completed with no failures.

Measure	Passed	Skipped	Failed	Total
Test suites	33	4	0	37
Individual tests	365	41	0	406

Skipped tests are reported separately and are **not** counted as passed. Their effect on coverage is discussed in Section 8.

6. Branch Coverage Results

6.1 Overall coverage

Branch coverage is the focus of this report. The other three metrics are given for context; they are analysed in the separate Code Coverage Report.

Metric	Covered / Total	Percentage	STP threshold (70%)
Branches	**466 / 483**	**96.48%**	Met
Statements	890 / 916	97.16%	Met
Functions	165 / 169	97.63%	Met
Lines	854 / 875	97.60%	Met

6.2 Module-wise branch coverage

Module	Branches covered	Branches total	Uncovered	Branch %
src	2	4	2	50%

Module	Branches covered	Branches total	Uncovered	Branch %
src/database	11	12	1	91.66%
src/middleware	31	31	0	100%
src/modules/admin	71	71	0	100%
src/modules/audit	9	9	0	100%
src/modules/auth	75	75	0	100%
src/modules/donations	143	152	9	94.07%
src/modules/donors	98	102	4	96.07%
src/modules/health	0	0	0	100% (no branches)
src/modules/stock	26	27	1	96.29%
Total	**466**	**483**	**17**	**96.48%**

Per-module counts sum to the overall 466 / 483. The `health` module has no branches, so its 100% reflects an empty denominator. The `src` row shows 50% because it contains only four branches; two misses make a large percentage swing in a very small sample.

6.3 Distribution of uncovered branches

Module	Uncovered branches	Share of the 17
src/modules/donations	9	53%
src/modules/donors	4	24%
src	2	12%
src/database	1	6%
src/modules/stock	1	6%

The donations module accounts for more than half of the gap, so it is the most efficient place to add tests if coverage is to be raised further.

7. Uncovered Branch Analysis

The table lists the files identified in the team's file-level coverage notes. These file-level percentages and line numbers come from the project's coverage run notes; they should be confirmed against the file pages in `docs/branch-coverage-html/` before final submission. Uncovered branches in this table are tests that were not written; they are not evidence of defects in the application.

File	Branch %	Location noted	Observation
app.js	50% (2/4)	Line 26 uncovered	Application bootstrap code; two of four branches not exercised.
donations.routes.js	81.81%	Line 24	Lowest file-level branch figure in the donations module.
donations.service.js	93.10%	Branches at lines 10, 29, 38, 57	Four decision points with one outcome unexercised.
donations.record.service.js	96.87%	Line 13	One remaining branch.

File	Branch %	Location noted	Observation
donorEligibility.js	91.11%	Lines 39, 72, 80	Eligibility rules (FR-DON-04, BR-03); worth targeting because the STP lists eligibility calculation as core business logic.
stock.service.js	93.33%	Lines 11-14 uncovered	One branch; the same lines are also uncovered for statements.

Not yet attributed to a file. The files above account for 5 of the 9 uncovered donations branches (derived from their percentages), 4 of 4 donors branches, 2 of 2 src branches and 1 of 1 stock branch. The remaining 4 donations branches and the 1 database branch are in files whose individual figures were not recorded in the notes. They can be identified from the HTML report.

Branch coverage versus other metrics. Two files behave differently from their branch figures. `rbac.js` has 100% branch coverage but only 91.66% statement coverage, and `transaction.js` has 100% branch coverage but low statement and function coverage, because its database execution paths are not exercised. A high branch percentage therefore does not by itself show that a file is thoroughly tested; it is reported alongside the other metrics for that reason.

8. Limitations

- **Database-dependent tests were skipped.** 41 tests in 4 suites did not run because no database was available. Coverage of the database layer is low (functions 0/3, statements 10/23), and branch coverage of code that relies on real queries and transactions reflects mocked behaviour only.
- **Mocked behaviour.** Passing unit tests with mocks show that the logic behaves correctly given the assumed responses of its dependencies. They do not show that the real database, SQL or transaction handling behaves the same way.
- **No integration or system evidence.** Atomic-transaction behaviour (TC-33), backup and restore (TC-34), performance (TC-17), TLS (TC-35) and browser compatibility (TC-36) are not demonstrated by this report. Do not read 96.48% branch coverage as complete database integration or end-to-end validation.
- **Coverage is not correctness.** Branch coverage shows that decision outcomes were executed, not that assertions on them are sufficient.
- **Mapping to test cases.** The suite is organised by module, not by STP test case ID (TC-01 to TC-37), so this report does not claim any specific STP test case as passed.

9. Conclusion

The backend unit test suite achieves **96.48% branch coverage (466 of 483 branches)**, with 97.16% statement, 97.63% function and 97.60% line coverage. All four values exceed the 70% unit-test coverage criterion in the STP (NFR-QUAL-04, TC-37 step 2). The run completed with 365 tests passed, 41 skipped and 0 failed.

Of the 17 uncovered branches, 9 are in the donations module and 4 in donors; the remaining gaps are small. The main limitation is that database-dependent tests were skipped, so the database layer and its transactional behaviour still require verification in a configured environment.

Recommended follow-up (optional, before submission):

- Add tests for the uncovered branches in `donations.service.js`, `donations.routes.js` and `donorEligibility.js`.
- Run the skipped database tests in an environment with PostgreSQL configured and re-run the coverage report.
- Confirm the file-level figures in Section 7 against the HTML report.

10. Supporting Evidence and Artifacts

Artifact	Location
HTML coverage report (entry page)	docs/branch-coverage-html/index.html
Original generated report	backend/coverage/lcov-report/index.html
Software Test Plan	docs/BBMS_Software_Test_Plan_v1.0.pdf
Test files added or modified	17 files under backend/src, committed and pushed to feature/18-record-donation (be6664c..60a5136)
Command	npm.cmd test -- --coverage

11. Review and Sign-off

Role	Name	Signature / Date
Test Engineer	Chukkapalli Rohan	
QA Lead	Dheepak Seetharam	
Product Owner	Harsimran Kaur	